ML Assignment 1

UNIVERSITY OF HARTFORD

Random Forest

Classification and Model Evaluation

Building a Random Forest classifier to predict mobile phone price ranges.

Student: John Bennett

Course: CS 569 Machine Learning

Fall 2025

September 25, 2025

Contents

1 Objective **1**

2 Data Set Evaluation and Cleaning **1**

3 Data Preparation **3**

4 Model Training **3**

5 Results and Discussion **5**

6 Conclusion **7**

1 Objective

In this assignment, I used a real-world dataset to build a Random Forest classifier to predict mobile phone price ranges in order to practice:

- Training a machine learning model.
- Evaluating its performance using accuracy, precision, recall, F1-score, and ROC/AUC.
- Visualizing evaluation results.

2 Data Set Evaluation and Cleaning

Table 1: Summary Statistics of Mobile Attributes

attribute	mean	std	min	25%	50%	75%	max	data type	number null	number unique
battery power	1238.518500	439.418206	501.000000	851.750000	1226.000000	1615.250000	1998.000000	int64	0	1094
blue	0.495000	0.500100	0.000000	0.000000	0.000000	1.000000	1.000000	int64	0	2
clock speed	1.522250	0.816004	0.500000	0.700000	1.500000	2.200000	3.000000	float64	0	26
dual sim	0.509500	0.500035	0.000000	0.000000	1.000000	1.000000	1.000000	int64	0	2
fc	4.309500	4.341444	0.000000	1.000000	3.000000	7.000000	19.000000	int64	0	20
four g	0.521500	0.499662	0.000000	0.000000	1.000000	1.000000	1.000000	int64	0	2
int memory	32.046500	18.145715	2.000000	16.000000	32.000000	48.000000	64.000000	int64	0	63
m dep	0.501750	0.288416	0.100000	0.200000	0.500000	0.800000	1.000000	float64	0	10
mobile wt	140.249000	35.399655	80.000000	109.000000	141.000000	170.000000	200.000000	int64	0	121
n cores	4.520500	2.287837	1.000000	3.000000	4.000000	7.000000	8.000000	int64	0	8
pc	9.916500	6.064315	0.000000	5.000000	10.000000	15.000000	20.000000	int64	0	21
px height	645.108000	443.780811	0.000000	282.750000	564.000000	947.250000	1960.000000	int64	0	1137
px width	1251.515500	432.199447	500.000000	874.750000	1247.000000	1633.000000	1998.000000	int64	0	1109
ram	2124.213000	1084.732044	256.000000	1207.500000	2146.500000	3064.500000	3998.000000	int64	0	1562
sc h	12.306500	4.213245	5.000000	9.000000	12.000000	16.000000	19.000000	int64	0	15
sc w	5.767000	4.356398	0.000000	2.000000	5.000000	9.000000	18.000000	int64	0	19
talk time	11.011000	5.463955	2.000000	6.000000	11.000000	16.000000	20.000000	int64	0	19
three g	0.761500	0.426273	0.000000	1.000000	1.000000	1.000000	1.000000	int64	0	2
touch$_s creen$	0.503000	0.500116	0.000000	0.000000	1.000000	1.000000	1.000000	int64	0	2
wifi	0.507000	0.500076	0.000000	0.000000	1.000000	1.000000	1.000000	int64	0	2
price range	1.500000	1.118314	0.000000	0.750000	1.500000	2.250000	3.000000	int64	0	4

The data set overall has 2000 rows with 21 columns with aggregate information Provided in Table 1.

Table 1 provides an overview of the contents of the dataset in its original form. Each attribute name is listed in the first column with summative statistics (including the mean, standard deviation, minimum value, quartile distribution, and maximum) of each attribute's values in successive columns, followed by the attribute's data type, number of null values, and number of unique values for each attribute in the remaining columns. This provides some inferences about each of the attributes and the role the will play in the training of the Random Forrest Model. The meaning of each attribute is as follows and can be verified at Kaggle:

battery_power Battery capacity of the mobile phone in mAh (milliampere-hour).

blue Indicates the presence of Bluetooth (`1` = yes, `0` = no).

clock_speed Processor speed in GHz. Higher values usually mean faster processing.

dual_sim Indicates whether the phone supports dual SIM cards (`1` = yes, `0` = no).

fc Front camera resolution in megapixels (MP). Higher values indicate better front camera quality.

four_g Indicates 4G network support (`1` = yes, `0` = no).

int_memory Internal memory (ROM) in GB. Higher values mean more storage space for apps, files, and media.

m_dep Mobile depth (thickness) in cm. Physically measures how thick the phone is.

mobile_wt Weight of the mobile phone in grams (g).

n_cores Number of processor cores. More cores can improve multitasking and performance.

pc Primary camera resolution in megapixels (MP). Higher values usually indicate better image quality.

px_height Pixel resolution along the height of the screen. Used with `px_width` to determine screen resolution.

px_width Pixel resolution along the width of the screen.

ram RAM size in MB. More RAM allows more applications to run simultaneously and improves performance.

sc_h Screen height in cm. Physical dimension of the display.

sc_w Screen width in cm. Physical dimension of the display.

talk_time Maximum talk time of the battery in hours.

three_g Indicates 3G network support (`1` = yes, `0` = no).

touch_screen Indicates if the phone has a touchscreen (`1` = yes, `0` = no).

wifi Indicates the presence of Wi-Fi support (`1` = yes, `0` = no).

price_range Target variable indicating the price category of the phone:

- `0` = low cost
- `1` = medium cost
- `2` = high cost
- `3` = very high cost

The table shows that a minimum amount of cleaning is necessary while giving clues to which attributes with numeric values are actually categorical data. There are no nulls in the data, meaning that either there are no unknown values or those values are coded in a different way. Of the attributes with low numbers of distinct values, the attributes with minimum/maximum values of 0/1, respectively, along with having only two unique values indicates that these attributes are actually binary categorical attributes, identifying the dataset's records having or lacking the feature indicated by the attribute. It must be assumed that these attributes (blue, dual sim, four g, three g, touch screen, and wifi) all have zero values that have meaning and are not indicators of unknown/incomplete information. Having eight unique values for the the number of cores logically indicates that values refer to a core count rather than a category but that it is a discrete value where the mean and standard deviation are less meaningful than the quartile values. The exception is the fact that the relatively close values of the mean and 50 percentile (median) indicate that the distribution of this attribute is relatively normal or uniform, suggesting a lack of outliers. Price range consists of four values indicating that this attribute is categorical and can be used as classes for each record. Having four distinct values suggests that zero is a class and not a placeholder.

Of the attributes with medium and highly distinct sets of values, several have minimum values of zero where it they are not consistent with what is being measured, indicating that they are not real values and are instead place holders. For example, the screen width (sc w) indicates the width of the screen so the zero values for this attribute have no meaning as there can't be a phone with no screen width. Attributes with this characteristic include front camera (fc), whose values are megapixels, pixel resolution for the height (px height), and screen width (sc w). Each of these were further evaluated for Null values where zero was replaced with NaN (PANDAS library's default numeric null type) and the overview table was re-evaluated.

The results of accurately updating zero values for the aforementioned attributes shows that front camera (fc) has 474 NaNs (24%), screen width (sc w) has 180 NaNs (9%), and pixel resolution for height (px height) has 2 NaNs (0.1%). In all cases the percentage is low enough to keep each attribute and instead impute values. The technique used for this is dot-decking using impute k-nearest neighbors from Scikit-Learn.

3 Data Preparation

Random Forest Classifier is a supervised model that uses training data to produce a model that classifies future data of unknown classification, provided the data being classified has the same attributes that trained the model. As random forest is a supervised classifier algorithm, the attribute that contains the classes being trained for needs to be distinguished from the attributes the data is trained on for the sake of validation measures and performance measures. The data was then further subset to establish training and testing datasets.This was achieved using train_test_split using Scikit-learn's model_selection module. The training data was set to 80% of the data and the testing set 20% of the data resulting in the former having 1600 data objects and the latter having 400.

4 Model Training

Random forests consist of a set of decision trees trained on the same pool of data that, when aggregated, provides a generalized model that eliminates the overfitting tendencies of each decision tree that informed on it. The decision trees that make up the random forest are generated by iteratively sub-setting the training dataset through random choosing of a set of attributes from it, which is then used to train a decision tree model. The resulting group of trees that define the random forest model are all used to classify unseen data. The model uses the frequency distribution of how often a data object is classified as each of the potential classifications in the class attribute to assign a probability score for unseen data objects being in the most frequent class.

The random forest has a number of hyperparameters that in combination provide an infinite set of possible constraints and chracteristics for the model being created. Because of this identifying good combinations hyperparameters consists of focusing on a set of hyperparameters to define (the rest have default values in scikit-learn's RandomForest algorithm), and a range of value that each parameter can take. There are many options for parameters shown in Table2.

Table 2: Hyperparameters of Random Forest Algorithms

Hyperparameter	**Function / Description**
n_estimators	Number of trees in the forest. More trees improve stability and reduce variance, but increase training time.
max_depth	Maximum depth of each tree. Limits how deep trees can grow to control overfitting; None means nodes expand until leaves are pure.
min_samples_split	Minimum number of samples required to split an internal node. Larger values prevent small splits and increase regularization.
min_samples_leaf	Minimum number of samples required to be at a leaf node. Ensures leaves have enough samples to avoid overfitting.
max_features	Number of features considered when looking for the best split. Controls randomness and diversity of trees. Common options: `sqrt`, `log2`, fraction of total features.
bootstrap	Whether samples are drawn with replacement when building trees. If False, the whole dataset is used for each tree.
criterion	Function to measure the quality of a split (e.g., `gini`, `entropy`). Determines how the tree decides splits.

The hyper-parameters that were chosen for training the model were the number of estimators (n_estimators), maximum tree depth (max_depth), and minimum data objects required to allocated to a node for that node to be a candidate for splitting (min_samples_split). In order to established an optimized value for each parameter a range of candidate values are created that then are tested in combinations in order select an ideal combination. This approach trains many different random forest models and then selects the best one based on a specified criteria. In this case GridSearchCV from Scikit-learn's model_selection module was used to navigate this approach. ROC-AUC was the measure used to perform validation of the random forest models fitted using a range of values for the selected hyper-paramters. This validation is for the model itself and is not a reflection of how well the model performs on the yet to be seen data in the test set but rather is utilized to optimize the hyper parameter values. The ranges were set dynamically for the maximum depth, and minimum sample split using the size of the training set. The output provides the best model as well as a table of all of the scores for the models trained with the top 5 shown in Table 3.

Table 3: Top Random Forest Hyperparameter Configurations

Parameters	Mean Test Score	Test-Train Gap
{'max_depth': 15, 'min_samples_split': 2, 'n_estimators': 500}	0.9786	0.0214
{'max_depth': 18, 'min_samples_split': 2, 'n_estimators': 500}	0.9783	0.0217
{'max_depth': 27, 'min_samples_split': 2, 'n_estimators': 500}	0.9783	0.0217
{'max_depth': 24, 'min_samples_split': 2, 'n_estimators': 500}	0.9783	0.0217
{'max_depth': 15, 'min_samples_split': 2, 'n_estimators': 300}	0.9782	0.0218

Table 3 indicates that the best values for the hyperparameters controlled for are a maximum tree depth of 15, a minimum number samples to split a node being 2, and the number of estimators being 500. This is achieved by one-versus-the-rest-cross validation of the tested models. The mean test score provides the main criteria for the choosing the hyperparameter value set and the Test-Train gap. This measure gap provides evidence for their being low potential for overfitting the model based on the model performing relatively similarly to the training model. A value greater than 0 indicates a better training score than testing score on the training set itself, as the sequestered testing set will only be utilized in performance measures on the model fitted through this process. The gap score of 0.0214 is indication that the model should generalize well which will be further clarified in the measure of the models performance on the unseen testing set.

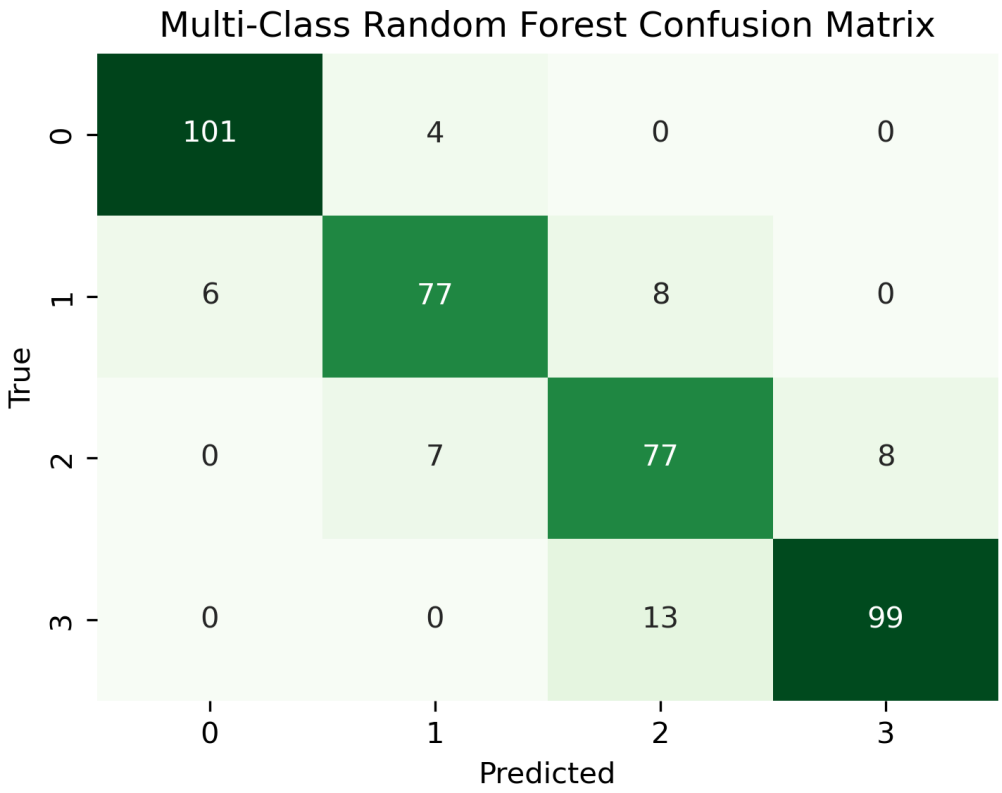
5 Results and Discussion

Table 4: Classification Report for Random Forest Model Price Predictions

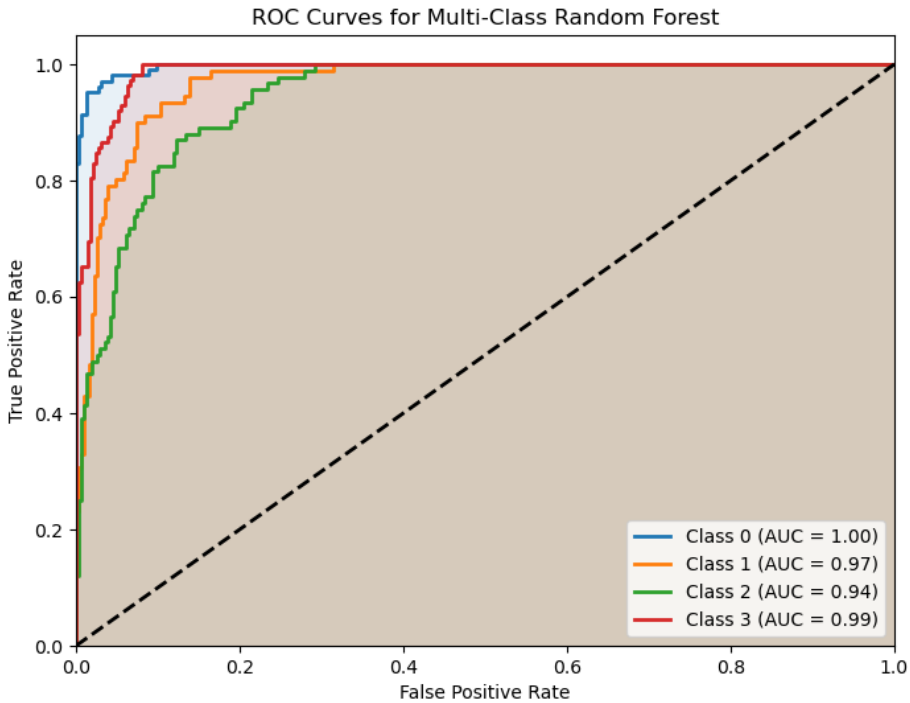
Class	Precision	Recall	F1-score	Support
0	0.94	0.96	0.95	105
1	0.87	0.84	0.85	91
2	0.79	0.84	0.81	92
3	0.93	0.89	0.91	112
Accuracy			0.89	400
Macro Avg	0.88	0.88	0.88	400
Weighted Avg	0.89	0.89	0.89	400

The model performed well overall with averages for scores close to 90%. The precision scores varied the most around for the for classification. They capture the proportion of accurate positive predictions to the total number of positive predictions. This shows the highest precision for class 0 which accounts for low cost phones, followed by High Cost Phones (class 3), then the next least expensive phone categories class 1, class 2). Recall again focuses on positive values where the proportion of true positives relative to all positives is measured. It showed the same sequenced of classes for its highest scores. The precision and recall are used to define the F1-score that, for each classification, combines the the models capacity to be consistent in categorizing objects correctly, as indicated in the precision measure, with

the measure of how frequently the model misses object of that class, or recall. It will also reflect the inverse where the model might not be precise but will be good at finding all of the object belonging to that class. It weights these measure the same, however, and there are instances of classification where you would want there to be better precision, like positively identifying an illness, or better recall like determining where loan defaults will occur. The Confusion Matrix below provides the particular breakdown of where the model effectively classified data objects and where it failed to do so.



The next graph provides the Reciever operating characteristic curves which indicates how effective the mode is at predicting the correct class. There are 4 curves ,one for each classification value. The model performed well for each class as there is a steep rate of increase for the true positive rate with nominal increase in the false positive rate. The area under each curve indicate the overall performance of the model, relative to the oblique midline indicating where a hypothetical model would be as good as random selection at classifying a data object correctly. The ideal value is the area would equal one , indicating the scenario where the model classifies all test data accurately which would appear as a complete rectangle. As the model approaches that outcome for each of the classes, the model overall very effective



6 Conclusion

Overall, the results of the fitted model performed well. There is potential for improvement. Possible mechanisms for improvement would be to use a larger data set of further fine tune the hyper-parameters by providing a better range of candidate parameter values. Another area to consider is the balance in the training data for the number of objects for each class though this is less of an issue for the random forest classifier than for other classification models.